Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Уфимский государственный нефтяной технический университет»

Кафедра «Цифровых технологий и моделирования»

Направление подготовки
09.04.01 Информатика и вычислительная техникаа
профиль «Искусственный интеллект и цифровые двойники в топливно-энергетическом комплексе»

Разработка системы диагностики нейродегенеративных заболеваний на основе анализа поведения пользователя в виртуальной среде

Пояснительная записка
к курсовой работе на 30 листах
по дисциплине «Нейровиртуальные технологии»

Выполнил:
студент гр. МПИ-09-25 А. А. Фарахьянов

Руководитель:
к.ф.-м.н., доцент Н. В. Чиганова

Уфа
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Анализ предметной области 4
1.1 Нейродегенеративные заболевания и особенности когнитивных нарушений 4
1.2 Электроэнцефалография в контексте исследования когнитивных нарушений 5
1.3 Методы обработки и анализа электроэнцефалографических сигналов 7
1.4 Использование технологий виртуальной реальности для исследования когнитивных функций 8
1.5 Обзор существующих подходов и обоснование выбора технологий 9
2 Разработка системы 11
2.1 Общая структура разрабатываемой системы 11
2.2 Проектирование виртуальной среды и экспериментального задания 12
2.3 Выбор и подготовка электроэнцефалографических данных 13
2.4 Вычисление спектральных признаков ЭЭГ 14
2.5 Разработка модели классификации ЭЭГ 16
2.6 Организация взаимодействия между Python и Unity 17
2.7 Объединение результатов и формирование отчета 17
2.8 Ограничения проектируемой системы 18
3 Реализация системы 19
3.1 Используемые средства разработки и программные технологии 19
3.2 Реализация виртуальной среды 20
3.3 Загрузка и обработка электроэнцефалографических записей 21
3.4 Реализация модели машинного обучения 22
3.5 Реализация программного интерфейса 23
3.6 Реализация взаимодействия Unity с API 25
3.7 Проверка работоспособности системы 26
ЗАКЛЮЧЕНИЕ 29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 30
ПРИЛОЖЕНИЕ А 31

ВВЕДЕНИЕ

Анализ ЭЭГ с применением цифровой обработки сигналов и машинного обучения позволяет исследовать когнитивные нарушения при нейродегенеративных заболеваниях. Дополнительные возможности даёт виртуальная реальность (VR): стандартизированные задания и регистрация поведения (время выполнения, ошибки). Совместное использование этих технологий актуально для создания исследовательских программных систем; результаты не заменяют клиническое обследование и не являются основанием для диагноза.

Цель – разработать программный прототип системы анализа ЭЭГ и регистрации поведения пользователя в VR с применением машинного обучения.

Задачи: изучить методы анализа ЭЭГ и подходы к исследованию когнитивных нарушений с использованием VR; обосновать выбор набора ЭЭГ-данных; разработать алгоритм извлечения признаков и классификации записей; реализовать интерфейс обработки записей и передачи результатов в Unity; разработать VR-среду с заданием на поиск товаров и регистрацией действий; реализовать визуализацию ЭЭГ, сохранение результатов и проверить работоспособность прототипа.

Объект исследования – методы компьютерного анализа ЭЭГ и регистрации поведения пользователя в VR.

Предмет исследования – методы извлечения частотных признаков ЭЭГ, классификации записей и интеграции результатов с показателями выполнения VR-задания.

Методы: цифровая обработка сигналов, спектральный анализ, машинное обучение, объектно-ориентированное программирование, проверка работоспособности ПО.

Практическая значимость – создание прототипа, объединяющего обработку и классификацию ЭЭГ, визуализацию сигнала, выполнение VR-задания и сохранение результатов в структурированном формате. Система предназначена для исследовательских целей и демонстрации интеграции указанных технологий.

1 Анализ предметной области

1.1 Нейродегенеративные заболевания и особенности когнитивных нарушений

Нейродегенеративные заболевания представляют собой группу патологических состояний, характеризующихся постепенным нарушением функционирования и гибелью нервных клеток. Развитие подобных заболеваний может сопровождаться изменениями памяти, внимания, мышления, речи, пространственной ориентации и способности выполнять повседневные действия. Выраженность и характер нарушений зависят от конкретного заболевания, стадии его развития и индивидуальных особенностей человека.

Одним из наиболее распространённых заболеваний данной группы является болезнь Альцгеймера. Она характеризуется прогрессирующим снижением когнитивных функций и является одной из основных причин деменции. Согласно опубликованным оценкам, на болезнь Альцгеймера приходится приблизительно 60–70 % случаев деменции[1].

Деменция представляет собой синдром, при котором наблюдается существенное ухудшение когнитивных способностей, влияющее на самостоятельность человека в повседневной жизни. При этом необходимо различать деменцию как клинический синдром и конкретные заболевания, которые могут её вызывать. Помимо болезни Альцгеймера, к причинам деменции относятся сосудистые заболевания головного мозга, лобно-височная дегенерация и деменция с тельцами Леви.

Исследование когнитивных нарушений предполагает использование различных методов оценки. К традиционным подходам относятся клиническое обследование, нейропсихологическое тестирование, анализ анамнеза и применение методов нейровизуализации. Дополнительные сведения могут предоставлять инструментальные методы исследования электрической активности мозга, а также компьютерные системы, регистрирующие выполнение стандартизированных заданий.

Особый интерес представляет автоматизация анализа данных. Применение алгоритмов машинного обучения позволяет выявлять закономерности в большом количестве измерений и строить модели, способные относить наблюдения к заранее определённым категориям. Однако качество подобных моделей зависит от объёма и состава обучающих данных, способа формирования признаков и корректности оценки результатов.

В рамках настоящей работы рассматривается разработка программного прототипа, объединяющего анализ электроэнцефалографических сигналов и регистрацию поведения пользователя в виртуальной среде. Такой подход позволяет продемонстрировать интеграцию инструментального анализа и регистрации количественных показателей выполнения заданий. Разрабатываемая система не предназначена для самостоятельной постановки медицинского диагноза.

1.2 Электроэнцефалография в контексте исследования когнитивных нарушений

Электроэнцефалография (ЭЭГ) представляет собой метод регистрации электрической активности головного мозга с помощью электродов, размещаемых на поверхности головы. Измеряемые сигналы отражают суммарную электрическую активность больших групп нейронов и изменяются во времени. Благодаря высокой временной разрешающей способности ЭЭГ применяется при исследовании функционального состояния мозга, диагностике эпилептических состояний и оценке ряда неврологических нарушений.

При исследовании когнитивных нарушений ЭЭГ представляет интерес как источник информации о характере электрической активности мозга. В научной литературе описаны изменения спектральных характеристик сигнала, связанные с болезнью Альцгеймера и другими формами деменции. К таким изменениям относятся замедление доминирующего ритма, увеличение относительной выраженности

медленных колебаний и снижение выраженности некоторых более высокочастотных компонентов [2; 3].

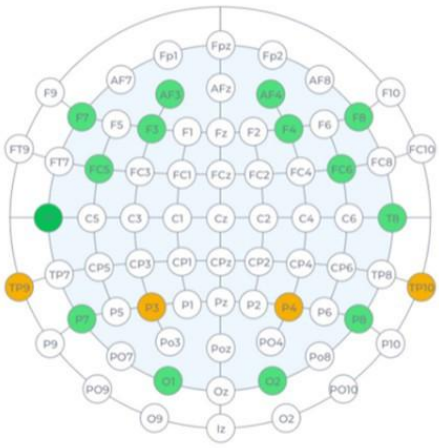
Для анализа ЭЭГ часто рассматриваются несколько частотных диапазонов:

- Дельта-диапазон – приблизительно от 1 до 4 Гц. Медленные колебания данного диапазона могут присутствовать в нормальной ЭЭГ, особенно во время сна, однако их выраженность и распределение зависят от состояния человека и условий регистрации.
- Тета-диапазон – от 4 до 8 Гц. Активность в этом диапазоне наблюдается в различных функциональных состояниях и исследуется в контексте процессов памяти и внимания.
- Альфа-диапазон – от 8 до 13 Гц. Альфа-активность особенно выражена в определённых условиях расслабленного бодрствования, например при закрытых глазах.
- Бета-диапазон – от 13 до 30 Гц. Данный диапазон связан с различными состояниями бодрствования и функциональной активностью нервной системы.
- Гамма-диапазон – приблизительно от 30 до 45 Гц в рамках рассматриваемого программного прототипа. Анализ этого диапазона требует учитывать возможное влияние мышечных артефактов и других помех.

Указанные границы являются условными и могут несколько различаться в зависимости от принятой методики. Частотный диапазон, используемый при обработке, также ограничивается характеристиками записи и выбранными параметрами анализа.

Для получения пространственно распределённых данных применяется система размещения электродов 10–20. Она определяет расположение электродов относительно анатомических ориентиров головы. Обозначения электродов позволяют связывать измерения с определёнными областями поверхности головы и обеспечивают единообразие размещения датчиков в различных исследованиях.

Система «10–20» или Международная система «10–20» – это общепризнанный во всем мире метод описания и размещения электродов на коже головы при проведении ЭЭГ-обследования, полисомнографического исследования сна или добровольных лабораторных исследований. Иллюстрация приведена на рис. 1:



Electrodes	Corresponding brain area
AF3/AF4	Left/right prefrontal cortex
F3/F4	Left/right frontal cortex
F7/F8	Left/right frontal cortex
FC5/FC6	Left/right primary motor area
T7/T8	Left/right temporal cortex
P7/P8	Left/right parietal cortex
O1/O2	Left/right occipital cortex

Рисунок 1 – Карта соответствия электродов и областей мозга

При анализе ЭЭГ важно учитывать, что изменения спектральных характеристик не являются специфичными исключительно для одного заболевания. На сигнал могут влиять возраст, состояние бодрствования, медикаментозная терапия, артефакты движений глаз и мышечной активности, характеристики оборудования и особенности предварительной обработки. Поэтому отдельный частотный показатель не может рассматриваться как достаточное основание для постановки диагноза.

В настоящей работе ЭЭГ используется как источник данных для разработки и демонстрации алгоритма классификации. Система анализирует цифровую запись, рассчитывает частотные признаки и передаёт их обученной модели машинного обучения. Полученный результат отражает решение модели относительно классов, представленных в обучающих данных.

1.3 Методы обработки и анализа электроэнцефалографических сигналов

ЭЭГ представляет собой многоканальный временной сигнал, содержащий полезную информацию и различные виды помех. Для его автоматизированного анализа необходимо преобразовать исходные измерения в форму, пригодную для последующей обработки алгоритмами машинного обучения.

Одним из распространённых подходов является спектральный анализ. Он позволяет оценить распределение мощности сигнала по частотам и определить, какие частотные компоненты преобладают в исследуемом интервале.

Для оценки спектральной плотности мощности может применяться метод Уэлча. При данном подходе сигнал разделяется на перекрывающиеся сегменты, для каждого из которых вычисляется периодограмма, после чего полученные оценки усредняются. Такое усреднение позволяет уменьшить дисперсию спектральной оценки по сравнению с анализом единственного сегмента.

1.4 Использование технологий виртуальной реальности для исследования когнитивных функций

Виртуальная реальность (Virtual Reality, VR) представляет собой интерактивную трёхмерную среду, позволяющую моделировать действия, приближённые к повседневной деятельности. В отличие от традиционных тестов на плоском экране, VR позволяет создавать задания на пространственную ориентацию, навигацию, запоминание расположения объектов и поиск предметов.

В научных исследованиях VR рассматривается как инструмент цифровой оценки когнитивных функций. При выполнении заданий можно регистрировать время, количество ошибок и последовательность действий пользователя [4]. Дополнительным преимуществом является возможность контролировать условия эксперимента: расположение объектов, правила взаимодействия и критерии завершения задания.

Однако результаты могут зависеть от опыта использования VR, особенностей управления и индивидуальных различий пользователей. Кроме того, показатели отдельного игрового задания не обладают самостоятельной диагностической специфичностью. Обзор исследований VR-ориентированных когнитивных заданий, опубликованный в 2026 году, отмечает перспективность данного направления, но указывает на необходимость дальнейшей проверки и стандартизации методов оценки [2].

В настоящей работе используется виртуальный супермаркет, в котором пользователь должен найти заданные товары. Программа регистрирует выбранные объекты, найденные предметы, ошибки и время выполнения. Данный сценарий

служит для сбора поведенческих данных и не рассматривается как самостоятельный диагностический тест.

1.5 Обзор существующих подходов и обоснование выбора технологий

Для разработки систем анализа ЭЭГ могут использоваться собственные записи, полученные с помощью специализированного оборудования, или открытые исследовательские наборы данных. Второй вариант позволяет разрабатывать и проверять алгоритмы без самостоятельной организации регистрации сигналов.

В настоящей работе выбран открытый набор OpenNeuro ds004504, содержащий записи 88 участников: 36 человек с болезнью Альцгеймера, 23 человека с лобно-височной деменцией и 29 участников контрольной группы. Записи получены в состоянии покоя с закрытыми глазами с использованием 19 электродов по системе 10–20 при частоте дискретизации 500 Гц.

Выбор набора обусловлен наличием нескольких диагностических групп, описанием условий регистрации и доступностью многоканальных ЭЭГ-записей. В качестве альтернативы рассматривался EEG Alzheimer's Dataset, однако недостаточная определённость происхождения данных, целевых меток и условий регистрации затрудняет их интерпретацию.

В текущей реализации используются две группы: AD – участники с болезнью Альцгеймера и CN – контрольная группа. Записи участников с лобно-височной деменцией исключены из обучения, поэтому классификатор решает задачу бинарного различения CN и AD.

Для обработки ЭЭГ выбран Python, предоставляющий библиотеки для анализа сигналов и машинного обучения. Виртуальная среда разрабатывается в Unity. Обмен данными между компонентами осуществляется через HTTP API: Python отвечает за обработку сигналов и классификацию, а Unity – за взаимодействие с пользователем, визуализацию и регистрацию поведения.

В текущей реализации анализ выполняется по заранее записанным ЭЭГ-файлам. Непрерывный сбор сигналов с электроэнцефалографического оборудования во время VR-задания не предусмотрен.

На основании рассмотренных подходов определены основные требования к прототипу:

- загрузка и обработка ЭЭГ-записей;
- вычисление спектральных признаков и классификация данных;
- передача и отображение результатов в Unity;
- реализация интерактивного VR-задания;
- регистрация поведенческих показателей;
- сохранение результатов в структурированном отчёте.

Сформулированные требования определяют функциональную основу системы и используются при её дальнейшем проектировании.

2 Разработка системы

2.1 Общая структура разрабатываемой системы

Разрабатываемая система предназначена для исследования возможности совместного анализа электроэнцефалографических данных и поведенческих характеристик пользователя, полученных при выполнении задания в виртуальной среде.

Система состоит из нескольких взаимодействующих компонентов. Виртуальная среда и регистрация действий пользователя реализованы в Unity, а обработка ЭЭГ и классификация данных выполняются на Python. Обмен информацией между ними осуществляется через HTTP API.

В состав системы входят:

- VR-модуль, обеспечивающий работу виртуального супермаркета и взаимодействие с объектами;
- модуль обработки ЭЭГ, выполняющий загрузку записей и вычисление спектральных признаков;
- модуль машинного обучения, определяющий класс анализируемой записи;
- API для передачи результатов анализа в Unity;
- модуль формирования отчёта, объединяющий результаты анализа ЭЭГ и показатели выполнения задания в CSV-файл.

Работа системы предусматривает выбор идентификатора участника, получение результатов анализа соответствующей записи ЭЭГ, выполнение задания в виртуальной среде и сохранение итоговых показателей. В текущей реализации используются заранее подготовленные записи ЭЭГ, поэтому подключение электроэнцефалографического оборудования непосредственно во время VR-задания не требуется.

Такая архитектура позволяет отдельно обрабатывать ЭЭГ и регистрировать поведение пользователя, а затем объединять полученные результаты в общем отчёте.

2.2 Проектирование виртуальной среды и экспериментального задания

В качестве виртуальной среды выбран супермаркет, разработанный в Unity с использованием XR Interaction Toolkit. Пользователю предлагается найти заданные товары на полках и выбрать их.

Подобный сценарий позволяет регистрировать поведенческие характеристики, связанные с поиском объектов, ориентированием в пространстве и принятием решений. При этом задание не является самостоятельным диагностическим тестом: его результаты используются как дополнительные показатели для последующего анализа.

В проектируемой сцене предусмотрены виртуальное помещение, полки с интерактивными товарами, список заданных предметов и интерфейс отображения состояния задания. В текущей реализации необходимо найти четыре объекта: молоко, хлеб, красное яблоко и приправу к рыбе.

Для оценки выполнения задания предусмотрена регистрация следующих показателей:

- времени выполнения;
- количества найденных предметов;
- общего количества выборов;
- количества ошибок;
- факта завершения задания.

Эти показатели позволяют количественно описать действия пользователя. Однако время поиска и число ошибок могут зависеть не только от когнитивных особенностей, но и от опыта работы с VR, навыков управления и индивидуальной стратегии. Поэтому их нельзя рассматривать как прямые признаки наличия заболевания.

2.3 Выбор и подготовка электроэнцефалографических данных

Для разработки и проверки модуля анализа ЭЭГ используется открытый набор данных OpenNeuro ds004504, содержащий записи электроэнцефалограммы пациентов с болезнью Альцгеймера, пациентов с лобно-височной деменцией и здоровых участников контрольной группы.

Набор данных включает записи, полученные в состоянии покоя с закрытыми глазами. Для регистрации сигналов использовалась система расположения электродов 10–20. В рассматриваемом проекте анализируются записи, содержащие 19 ЭЭГ-каналов. Частота дискретизации исходных записей составляет 500 Гц. Описание набора данных опубликовано в научной литературе и доступно в репозитории OpenNeuro.

Для обработки сигналов применяется язык программирования Python. Загрузка и обработка файлов формата EEGLAB .set выполняются с использованием библиотеки MNE-Python. Для вычисления спектральных характеристик применяется функция оценки спектральной плотности мощности из библиотеки SciPy, а обучение классификатора выполняется средствами Scikit-learn.

Перед вычислением признаков выполняются следующие операции:

- Загрузка записи ЭЭГ из файла.
- Выбор электроэнцефалографических каналов.
- Определение частоты дискретизации и длительности записи.
- Вычисление спектральной плотности мощности сигнала.
- Определение мощности в заданных частотных диапазонах.
- Формирование вектора признаков фиксированной длины.

Использование записей из открытого набора данных позволяет проводить эксперименты без необходимости самостоятельного сбора клинических электроэнцефалографических сигналов. При этом необходимо учитывать, что исходные записи получены в условиях, отличающихся от выполнения задания в виртуальной среде. Поэтому результаты классификации ЭЭГ и показатели поведения

пользователя в текущей реализации представляют собой два отдельных источника информации, объединяемых в итоговом отчёте.

2.4 Вычисление спектральных признаков ЭЭГ

Электроэнцефалограмма представляет собой временной ряд, отражающий изменение электрической активности мозга. Для автоматизированного анализа исходный сигнал преобразуется в набор числовых характеристик, которые могут использоваться алгоритмом машинного обучения.

В данной работе основным способом представления сигнала являются спектральные признаки, характеризующие распределение мощности по частотным диапазонам. Для их получения используется оценка спектральной плотности мощности методом Уэлча.

Метод Уэлча предусматривает разделение сигнала на перекрывающиеся сегменты, применение оконной функции, вычисление периодограмм отдельных сегментов и последующее усреднение полученных оценок. Такой подход позволяет получить более устойчивую оценку спектральной плотности мощности по сравнению с использованием периодограммы всего сигнала без усреднения.

Спектральная плотность мощности обозначается функцией (P(f)), где (f) – частота. Для каждого электроэнцефалографического канала рассчитывается средняя мощность в пяти частотных диапазонах:

Таблица 2.1 – Частотные диапазоны

Диапаозн	**Частотные границы**
Дельта	1-4 Гц
Тета	4-8 Гц
Альфа	8-13 Гц
Бета	13-30 Гц
Гамма	30-45 Гц

Средняя спектральная мощность для выбранного частотного диапазона может быть представлена выражением:

$$P_B = \frac{1}{N_B} \sum_{f_k \in B} P(f_k),$$

где B – рассматриваемый частотный диапазон, $P(f_k)$ – оценка спектральной плотности мощности на частоте f_k , а N_B – количество частотных отсчётов, попадающих в диапазон.

В реализованном алгоритме для уменьшения влияния широкого диапазона значений мощности применяется логарифмическое преобразование:

$$F_B = log_{10}(max(P_B, \varepsilon)),$$

где ε – небольшая положительная константа, предотвращающая вычисление логарифма нуля.

Таким образом, для каждого канала формируется пять признаков, соответствующих выбранным частотным диапазонам. При использовании 19 каналов общее количество признаков составляет:

$$N_{features} = N_{channels} \cdot N_{bands},$$

$$N_{features} = 19 \cdot 5 = 95.$$

Следовательно, каждая запись ЭЭГ представляется в виде вектора из 95 числовых признаков. Такой формат позволяет подавать данные на вход модели машинного обучения независимо от исходной длительности записи при условии, что структура каналов и порядок признаков соответствуют требованиям обученной модели.

Следует отметить, что полученные признаки характеризуют спектральное распределение мощности ЭЭГ, но не описывают все возможные свойства сигнала. В частности, в текущей реализации не выполняется комплексный анализ временных зависимостей, связности между каналами или динамических изменений спектра в ходе когнитивного задания.

Графики изменения спектральной мощности ЭЭГ во времени для пяти каналов и пяти частотных диапазонов с эмулированного шлема Virtual Brainwear приведены на рис. 2.



Рисунок 2 – Данные с 5 каналов шлема INSIGHT

2.5 Разработка модели классификации ЭЭГ

Для классификации электроэнцефалографических записей используется алгоритм случайного леса (Random Forest), относящийся к ансамблевым методам машинного обучения. Он объединяет результаты нескольких деревьев решений и подходит для обработки табличных данных, а также позволяет получать оценки принадлежности объектов к классам.

В текущей реализации выполняется бинарная классификация: CN – контрольная группа, AD – участники с болезнью Альцгеймера. Записи участников с лобно-височной деменцией (FTD) в обучение модели не включаются.

Входными данными служат 95 спектральных признаков. Модель реализована с помощью Scikit-learn и использует следующие параметры:

- количество деревьев – 300;
- балансировка весов классов – `balanced`;
- минимальное количество объектов в листе – 2;
- `random_state` – 42.

Обученная модель сохраняется в файл `eeg_model.joblib` вместе с метаданными о составе и порядке признаков, частотных диапазонах и классах. Для оценки качества классификации используются сбалансированная точность и перекрёстная проверка. Сбалансированная точность представляет собой среднее значение полноты по классам и учитывает возможный дисбаланс выборки.

2.6 Организация взаимодействия между Python и Unity

Взаимодействие модуля анализа ЭЭГ и VR-приложения организовано по клиент-серверной архитектуре. Серверная часть реализована на Python с использованием FastAPI, а Unity выступает клиентом, отправляющим HTTP-запросы.

Программный интерфейс предоставляет три конечные точки: GET /health для проверки доступности сервера, POST /predict для классификации переданных признаков и GET /predict_file?participant_id=sub-001 для анализа записи ЭЭГ по идентификатору участника.

При запросе анализа сервер загружает соответствующую запись, вычисляет спектральные признаки и передаёт их обученной модели. Результат возвращается в формате JSON и содержит идентификатор участника, предсказанный класс, оценки вероятности для CN и AD, количество признаков и каналов, длительность записи, а также нормализованные данные ЭЭГ для визуализации.

На стороне Unity запросы обрабатывает компонент EEGApiClient.cs, а отображение сигнала выполняется с помощью EEGGraphUI. Сервер запускается локально по адресу http://127.0.0.1:8000. Такая архитектура позволяет независимо изменять алгоритмы анализа и VR-интерфейс и подходит для экспериментального прототипа.

2.7 Объединение результатов и формирование отчета

Для объединения результатов анализа ЭЭГ и выполнения VR-задания предусмотрен компонент ExperimentReportManager.cs. Он формирует общую запись,

содержащую идентификатор участника, результаты классификации, оценки вероятности классов CN и AD, количество признаков и каналов, длительность записи ЭЭГ, а также показатели выполнения задания: количество найденных предметов, выборов и ошибок, время выполнения и факт завершения.

Результаты сохраняются в формате CSV: каждая строка соответствует отдельному эксперименту, а столбцы содержат зарегистрированные показатели. Файлы размещаются в каталоге ExperimentReports внутри директории Application.persistentDataPath.

Объединение выполняется по идентификатору участника, что позволяет связать результаты разных программных компонентов. При этом отчёт представляет собой совместное хранение показателей ЭЭГ и VR-задания, а не результат их классификации единой моделью.

2.8 Ограничения проектируемой системы

Система анализирует заранее записанные данные ЭЭГ, полученные в состоянии покоя и не синхронизированные с действиями пользователя в виртуальной среде. Поэтому непосредственная связь между поведением и изменениями электроэнцефалографической активности в текущей реализации не устанавливается.

Классификатор различает только два класса — CN и AD — и не предназначен для определения других типов нейродегенеративных заболеваний. Качество классификации зависит от состава выборки и методики оценки, поэтому важны разделение данных по участникам и проверка на независимой тестовой выборке. На поведенческие показатели могут влиять опыт использования VR, особенности управления и индивидуальная стратегия выполнения задания. Система является исследовательским прототипом и не предназначена для самостоятельной медицинской диагностики.

3 Реализация системы

3.1 Используемые средства разработки и программные технологии

Для реализации системы использовались Unity и C# для разработки виртуальной среды, а также Python для обработки ЭЭГ и машинного обучения.

Виртуальное взаимодействие организовано с помощью XR Interaction Toolkit. Для обработки электроэнцефалографических записей применяются MNE-Python и SciPy, для обучения и использования классификатора – Scikit-learn, для сохранения модели – Joblib. Обмен данными между Unity и Python реализован с помощью FastAPI. Основные используемые технологии представлены в таблице 3.1.

Таблица 3.1 – Используемые технологии

Технология	**Назначение**
Unity	Разработка виртуальной среды
C#	Реализация логики VR-приложения
XR Interaction Toolkit	Организация взаимодействия с объектами виртуальной сцены
Python	Реализация обработки ЭЭГ и классификации
MNE-Python	Загрузка и обработка файлов ЭЭГ
SciPy	Вычисление спектральной плотности мощности
Scikit-learn	Обучение и применение модели машинного обучения
Joblib	Сохранение обученной модели
FastAPI	Предоставление программного интерфейса для взаимодействия компонентов
CSV	Сохранение признаков и результатов эксперимента

Перечисленные средства образуют программную основу разработанной системы. Их совместное использование обеспечивает выполнение основных функций прототипа: обработку ЭЭГ, классификацию данных, взаимодействие с виртуальной средой и сохранение результатов.

3.2 Реализация виртуальной среды

Виртуальная сцена супермаркета реализована в Unity. Для организации XR-взаимодействия используется компонент XR Origin, обеспечивающий представление положения пользователя и работу механизмов взаимодействия с объектами сцены.

Логика задания реализована в компоненте ShopTaskManager. Он проверяет выбранные пользователем предметы, учитывает правильные и ошибочные действия, подсчитывает найденные объекты и фиксирует время выполнения задания. При выборе обязательного предмета он засчитывается при условии, что ранее не был найден.

Последовательность этапов исследования контролируется компонентом StudyPhaseManager, который обеспечивает переход между предусмотренными фазами работы приложения.

После завершения задания сформированные показатели передаются модулю формирования отчёта. Таким образом, реализация VR-модуля обеспечивает выполнение задания и регистрацию поведенческих данных для дальнейшего анализа. Фрагмеент геймплея во время фазы исследованяи приведён на рис. 3.



Рисунок 3 – Фрагмент геймплея

3.3 Загрузка и обработка электроэнцефалографических записей

Для работы с ЭЭГ используется набор данных OpenNeuro ds004504. Записи хранятся в формате EEGLAB .set, который поддерживается библиотекой MNE-Python.

Перед обработкой пользователь или оператор указывает идентификатор участника. По этому идентификатору сервер находит соответствующий файл в локальной копии набора данных. После загрузки записи выбираются электроэнцефалографические каналы, необходимые для вычисления признаков.

В рамках текущей реализации используется 19 каналов ЭЭГ. Для каждого канала вычисляется спектральная плотность мощности методом Уэлча. Параметр длины сегмента устанавливается равным четырём секундам сигнала:

$$N_{segment} = 4f_s,$$

где f_s – частота дискретизации сигнала.

При частоте дискретизации 500 Гц длина сегмента составляет:

$$N_{segment} = 4 \cdot 500 = 2000$$

отсчётов.

Далее для каждого канала определяется средняя мощность в пяти частотных диапазонах: дельта, тета, альфа, бета и гамма. Полученные значения преобразуются логарифмически и включаются в итоговый вектор признаков.

Алгоритм обработки записи можно представить в виде следующей последовательности:

- Получение идентификатора участника.
- Поиск соответствующего файла ЭЭГ.
- Загрузка записи средствами MNE-Python.
- Выбор электроэнцефалографических каналов.
- Вычисление спектральной плотности мощности.
- Расчёт мощности в пяти частотных диапазонах.
- Логарифмическое преобразование значений.
- Формирование вектора из 95 признаков.
- Передача признаков обученной модели.

Следует учитывать, что вычисление спектральных признаков само по себе не гарантирует устранения всех артефактов записи. В частности, текущий алгоритм не включает полноценную процедуру автоматического удаления глазных, мышечных и иных артефактов. Поэтому качество исходных записей остаётся одним из факторов, влияющих на результаты классификации.

3.4 Реализация модели машинного обучения

Для классификации записей ЭЭГ реализована модель Random Forest с использованием библиотеки Scikit-learn. Обучение выполняется на основе векторов, содержащих 95 спектральных признаков, и предусматривает разделение данных на два класса: CN и AD.

При обучении используются 300 деревьев решений, сбалансированные веса классов (class_weight="balanced"), минимальное количество объектов в листе дерева, равное двум (min_samples_leaf=2), и фиксированное значение генератора случайных чисел (random_state=42).

После завершения обучения модель сохраняется в файл eeg_model.joblib. Вместе с ней сохраняются метаданные, содержащие информацию о составе и порядке признаков, используемых частотных диапазонах и названиях классов. Это обеспечивает соответствие структуры входных данных требованиям обученной модели при последующей классификации.

При обработке запроса сервер загружает сохранённую модель и передаёт ей сформированный вектор признаков. В результате классификатор возвращает предсказанный класс и оценки вероятности принадлежности к каждой из двух категорий. Полученные значения преобразуются в структуру данных для дальнейшей передачи в приложение Unity.

Следует учитывать, что оценки вероятности, предоставляемые классификатором, отражают результат его работы и не являются клинической оценкой вероятности наличия заболевания. Разработанная модель используется исключительно в исследовательских целях и не предназначена для самостоятельной медицинской диагностики.

3.5 Реализация программного интерфейса

Для обеспечения взаимодействия между Python и Unity разработан программный интерфейс на основе FastAPI. Сервер принимает HTTP-запросы, выполняет необходимые операции и возвращает результаты в формате JSON.

В реализации предусмотрены три основные конечные точки.

Таблица 3.2 – Конечные точки API

Метод	Конечная точка	Назначение
GET	/health	Проверка доступности сервера
POST	/predict	Классификация переданного набора признаков
GET	/predict_file?participant_id=sub-001	Анализ записи ЭЭГ по идентификатору участника

Конечная точка /health используется для проверки того, что сервер запущен и доступен для обработки запросов.

Конечная точка /predict предназначена для классификации уже сформированного набора признаков. Такой подход позволяет разделить вычисление признаков и применение модели, а также использовать классификатор в других программных компонентах.

Конечная точка /predict_file реализует более полный сценарий обработки. Она получает идентификатор участника, находит соответствующую запись ЭЭГ, вычисляет признаки и выполняет классификацию. После этого сервер формирует ответ, содержащий идентификатор участника, результат классификации, вероятности классов, количество признаков, количество каналов и длительность записи.

Дополнительно ответ включает данные электроэнцефалографического канала для визуализации временного сигнала в интерфейсе Unity.

Для локального запуска сервера используется команда:

python -m uvicorn eeg_api:app --host 127.0.0.1 --port 8000

После запуска API доступен по локальному адресу http://127.0.0.1:8000. Для просмотра доступных методов и тестирования запросов можно использовать автоматически формируемую документацию FastAPI по адресу http://127.0.0.1:8000/docs. Документация представлена на рис. 3.

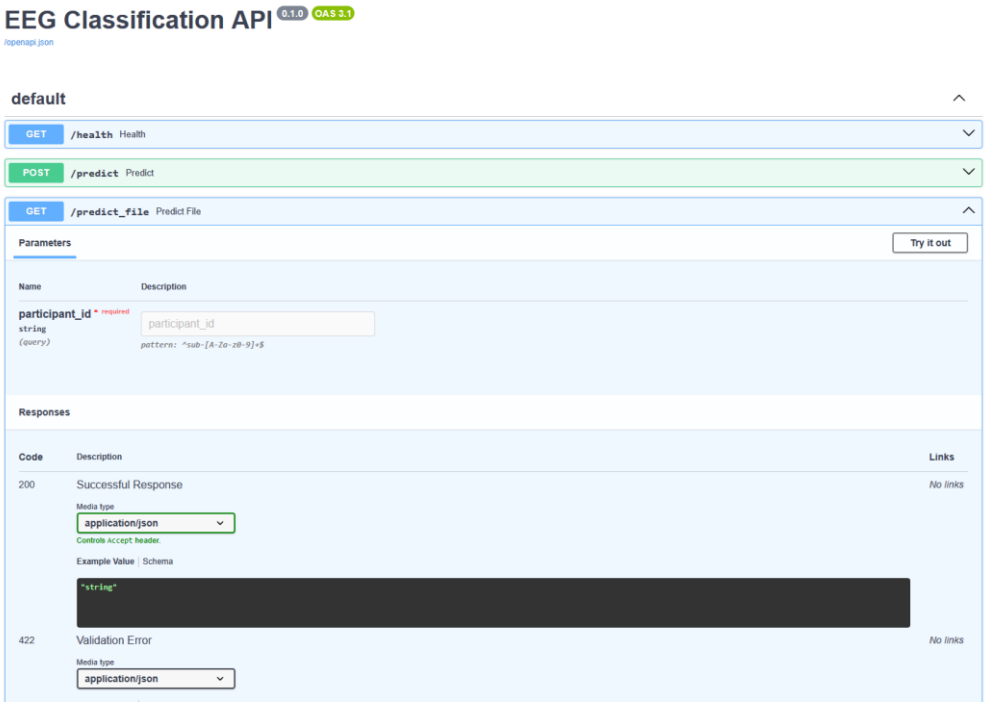


Рисунок 4 – Документация к EEG API

Локальный запуск позволяет выполнять все вычисления на одном компьютере и не требует организации внешнего сервера. Однако такой вариант подходит преимущественно для разработки и экспериментального использования.

3.6 Реализация взаимодействия Unity с API

На стороне Unity взаимодействие с сервером реализовано в компоненте EEGApiClient.cs. Компонент формирует запрос с идентификатором участника, отправляет его на сервер и обрабатывает полученный ответ.

После получения JSON-ответа приложение извлекает необходимые поля: идентификатор участника, предсказанный класс, оценки вероятностей, количество признаков, количество каналов и длительность записи. Эти значения передаются соответствующим элементам пользовательского интерфейса.

Пользователь может указать идентификатор участника, для которого необходимо выполнить анализ. После отправки запроса приложение ожидает ответ сервера и отображает полученный результат.

Такой механизм позволяет выполнять классификацию без повторного обучения модели. Обучение производится заранее, а во время работы приложения используется сохранённая модель.

Для визуального представления ЭЭГ применяется компонент EEGGraphUI. Он получает массив значений временного сигнала и отображает его в виде графика. Передаваемые значения нормализованы, поэтому график предназначен прежде всего для визуального отображения формы сигнала, а не для непосредственного измерения его амплитуды в физических единицах.

В результате пользовательский интерфейс объединяет текстовую информацию о результате классификации с графическим представлением ЭЭГ. Это повышает наглядность работы прототипа и позволяет демонстрировать основные этапы обработки данных.

При возникновении ошибки запроса или недоступности сервера приложение должно информировать пользователя о невозможности получить результат. Для повышения надёжности при дальнейшем развитии системы целесообразно также предусмотреть контроль корректности идентификатора, обработку отсутствующих файлов и проверку структуры ответа сервера.

3.7 Проверка работоспособности системы

Проверка прототипа проводилась поэтапно. На первом этапе проверялась возможность загрузки записи ЭЭГ и выполнения классификации серверным модулем. На втором этапе проверялось взаимодействие Unity с программным интерфейсом. На третьем этапе проверялась регистрация результатов VR-задания и сохранение итогового отчёта.

При тестировании использовалась запись участника с идентификатором sub-001. Сервер успешно загрузил соответствующую запись, вычислил 95 признаков на основе 19 каналов и вернул результат классификации.

В одном из выполненных тестов модель отнесла запись к классу AD. Полученные оценки составили приблизительно 0,1619 для класса CN и 0,8381 для

класса AD. Эти значения характеризуют вывод модели для конкретной записи и не являются подтверждением медицинского диагноза.

В ходе проверки также было подтверждено получение данных для отображения графика ЭЭГ. Продолжительность анализируемой записи составила приблизительно 599,8 секунды.

Отдельно проверялась работа виртуального супермаркета. Пользователь смог выбрать все четыре обязательных предмета, после чего приложение зарегистрировало завершение задания и передало показатели в модуль формирования отчёта.

В итоговом отчёте тестового запуска были зарегистрированы следующие значения:

Таблица 3.3 – Пример результата тестирования

Показатель	**Полученное значение**
Идентификатор участника	sub-001
Анализ ЭЭГ завершён	Да
Предсказанный класс	AD
Оценка для класса CN	0,1619
Оценка для класса AD	0,8381
Количество признаков ЭЭГ	95
Количество каналов	19
Длительность записи ЭЭГ	599,8 с
VR-задание завершено	Да
Найдено предметов	4 из 4
Зарегистрировано выборов	4
Время выполнения VR-задания	14,4 с

Полученные результаты подтверждают работоспособность основных программных компонентов и возможность объединения их выходных данных в едином отчёте. Однако данный тест является проверкой функционирования программного прототипа, а не полноценным клиническим исследованием или статистическим подтверждением диагностической эффективности системы.

Для более полной оценки необходимо провести серию повторных испытаний, проверить корректность регистрации ошибок и времени выполнения, а также оценить качество классификации на независимой выборке участников.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработан программный прототип системы для исследования нейродегенеративных заболеваний на основе анализа ЭЭГ и регистрации поведения пользователя в виртуальной среде. Были рассмотрены методы спектрального анализа электроэнцефалографических данных, применение машинного обучения и возможности VR для оценки поведенческих характеристик. На основе проведённого анализа спроектирована архитектура системы, объединяющей виртуальную среду, обработку ЭЭГ, классификацию данных и формирование отчётов.

В практической части разработана виртуальная среда супермаркета в Unity с заданием на поиск товаров. Для анализа записей из набора OpenNeuro ds004504 на Python реализовано вычисление 95 спектральных признаков (19 каналов и 5 частотных диапазонов) и обучена модель Random Forest для различения контрольной группы и группы болезни Альцгеймера. Взаимодействие с Unity организовано через FastAPI, а результаты классификации и показатели выполнения VR-задания сохраняются в CSV-отчётах. Проверка работоспособности подтвердила выполнение основных функций прототипа. Таким образом, поставленная цель достигнута.

Разработанная система имеет ограничения: анализ выполняется на заранее записанных данных ЭЭГ, классификатор различает только два класса, а поведенческие и электроэнцефалографические показатели пока не используются совместной моделью. Дальнейшее развитие предполагает проверку классификатора на независимой выборке, расширение набора поведенческих признаков и синхронизацию регистрации ЭЭГ с действиями пользователя в VR. Прототип может служить основой дальнейших исследований, однако не предназначен для самостоятельной медицинской диагностики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. 2023 Alzheimer's disease facts and figures. – DOI 10.1002/alz.13016 // Alzheimer's & Dementia. – 2023. – Vol. 19, № 4. – P. 1598–1695
2. The EEG analysis and identification of Alzheimer's disease: a review / [Автор 1], [Автор 2], [Автор 3] [и др.] // Frontiers in Aging Neuroscience. – 2025. – Vol. 17. – Art. 1686628. – DOI 10.3389/fnagi.2025.1686628. – URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC12722947/ (дата обращения: 20.10.2026).
3. Hort J., O'Brien J. T., Gainotti G. [и др.]. EFNS guidelines for the diagnosis and management of Alzheimer's disease // European Journal of Neurology. – 2010. – Vol. 17, № 10. – P. 1236–1248. – DOI 10.1111/j.1468-1331.2010.03040.x.
4. Bhargava Y., Tripathi S., Baths V., Deshpande B. M. Navigation performance in virtual reality–based cognitive assessment games as a candidate digital cognitive marker of hippocampal vulnerability: a scoping review // Alzheimer's & Dementia: Diagnosis, Assessment & Disease Monitoring. – 2026. – Vol. 18, № 3. – Art. e70434. – DOI 10.1002/dad2.70434. – URL: https://alz-journals.onlinelibrary.wiley.com/doi/10.1002/dad2.70434 (дата обращения: 22.10.2026).
5. Miltiadous A. et al. A Dataset of Scalp EEG Recordings of Alzheimer's Disease, Frontotemporal Dementia and Healthy Subjects from Routine EEG // Data. – 2023. – Vol. 8, No. 6. – Article 95. – DOI: 10.3390/data8060095.
6. OpenNeuro. A Dataset of Scalp EEG Recordings of Alzheimer's Disease, Frontotemporal Dementia and Healthy Subjects from Routine EEG: ds004504 [Электронный ресурс]. – URL: OpenNeuro ds004504 (дата обращения: 30.10.2026).
7. Gramfort A. et al. MEG and EEG data analysis with MNE-Python // Frontiers in Neuroscience. – 2013. – Vol. 7. – Article 267. – DOI: 10.3389/fnins.2013.00267.
8. Virtanen P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python // Nature Methods. – 2020. – Vol. 17. – P. 261–272. – DOI: 10.1038/s41592-019-0686-2.
9. Pedregosa F. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.
10. Breiman L. Random Forests // Machine Learning. – 2001. – Vol. 45. – P. 5–32. – DOI: 10.1023/A:1010933404324.
11. Unity Technologies. Unity Manual [Электронный ресурс]. – URL: Unity Manual (дата обращения: 30.10.2026).
12. Unity Technologies. XR Interaction Toolkit [Электронный ресурс]. – URL: XR Interaction Toolkit Documentation (дата обращения: 30.10.2026).
13. FastAPI. FastAPI Documentation [Электронный ресурс]. – URL: FastAPI Documentation (дата обращения: 30.10.2026).
14. Joblib. Joblib Documentation [Электронный ресурс]. – URL: Joblib Documentation (дата обращения: 30.10.2026).

ПРИЛОЖЕНИЕ А

check_eeg_data.py:

```
import pandas as pd
import numpy as np

# Путь к CSV-файлу
path = "my_eeg_csv_test_INSIGHT.csv"

# Первая строка содержит метаданные, поэтому пропускаем её
df = pd.read_csv(path, skiprows=1)

channels = ["AF3", "T7", "Pz", "T8", "AF4"]
bands = ["Theta", "Alpha", "BetaL", "BetaH", "Gamma"]

print("=" * 70)
print("1. ОБЩАЯ ИНФОРМАЦИЯ")
print("=" * 70)

print("Размер таблицы:", df.shape)
print("Названия столбцов:")
print(df.columns.tolist())

# Проверяем длительность записи
if "Timestamp" in df.columns:
    timestamps = pd.to_numeric(df["Timestamp"], errors="coerce").dropna()

    if len(timestamps) > 1:
        duration = timestamps.iloc[-1] - timestamps.iloc[0]
        print(f"\nДлительность записи: {duration:.2f} секунд")

print("\n" + "=" * 70)
print("2. СЫРЫЕ EEG-СИГНАЛЫ")
print("=" * 70)

for channel in channels:
    channel_str = "EEG."+channel
    if channel_str not in df.columns:
        print(f"\nКанал {channel_str}: столбец не найден")
        continue

    signal = pd.to_numeric(df[channel_str], errors="coerce").dropna()

    print(f"\nКанал {channel_str}:")
    print(f"  Количество значений: {len(signal)}")
    print(f"  Среднее:             {signal.mean():.3f}")
    print(f"  Стандартное отклонение: {signal.std():.3f}")
    print(f"  Минимум:             {signal.min():.3f}")
    print(f"  Максимум:            {signal.max():.3f}")
    print(f"  Медиана:             {signal.median():.3f}")
    print(f"  Пропусков:           {df[channel_str].isna().sum()}")

    # Центрирование нужно только для анализа колебаний
    centered = signal - signal.mean()

    print(f"  После вычитания среднего:")
    print(f"    Минимум:           {centered.min():.3f}")
    print(f"    Максимум:          {centered.max():.3f}")
    print(f"    Среднеквадратичное отклонение: {centered.std():.3f}")

print("\n" + "=" * 70)
print("3. МОЩНОСТЬ EEG ПО ДИАПАЗОНАМ ЧАСТОТ")
print("=" * 70)
```

```
for channel in channels:
    print(f"\n--- Канал {channel} ---")

    for band in bands:
        column = f"POW.{channel}.{band}"

        if column not in df.columns:
            print(f"{band}: столбец не найден")
            continue

        values = pd.to_numeric(df[column], errors="coerce").dropna()

        if len(values) == 0:
            print(f"{band}: нет значений")
            continue

        print(
            f"{band:6s} | "
            f"N={len(values):3d} | "
            f"среднее={values.mean():9.4f} | "
            f"медиана={values.median():9.4f} | "
            f"std={values.std():9.4f} | "
            f"min={values.min():9.4f} | "
            f"max={values.max():9.4f}"
        )

print("\n" + "=" * 70)
print("4. КОРРЕЛЯЦИИ МЕЖДУ СЫРЫМИ EEG-КАНАЛАМИ")
print("=" * 70)

available_channels = [ch for ch in channels if ch in df.columns]

eeg = df[available_channels].apply(pd.to_numeric, errors="coerce")
print(eeg.corr().round(3).to_string())

print("\n" + "=" * 70)
print("5. ПЕРВЫЕ 10 СТРОК POWER-ДАННЫХ")
print("=" * 70)

power_columns = [
    f"POW.{channel}.{band}"
    for channel in channels
    for band in bands
    if f"POW.{channel}.{band}" in df.columns
]

if power_columns:
    cols_to_show = (
        ["Timestamp"] if "Timestamp" in df.columns else []
    ) + power_columns

    print(df[cols_to_show].head(10).to_string(index=False))
else:
    print("Столбцы POW не найдены.")

print("\nАнализ завершён.")
```

eeg_api.py:

```
import numpy as np
import mne

from scipy.signal import welch
from fastapi import Query

from pathlib import Path

import joblib
import pandas as pd

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "eeg_model.joblib"
FEATURES_PATH = BASE_DIR / "eeg_features.csv"

bundle = joblib.load(MODEL_PATH)
model = bundle["model"]
feature_columns = bundle["feature_columns"]
classes = bundle["classes"]

DATASET_DIR = Path(r"C:\Users\faral\Downloads\v1.0.0")
DERIVATIVES_DIR = DATASET_DIR / "derivatives"

BANDS = {
    "delta": (1, 4),
    "theta": (4, 8),
    "alpha": (8, 13),
    "beta": (13, 30),
    "gamma": (30, 45),
}
# Готовые признаки для демонстрации
features_df = pd.read_csv(FEATURES_PATH)

app = FastAPI(title="EEG Classification API")


class PredictionRequest(BaseModel):
    sample_index: int = 0

def extract_features(filepath):
    raw = mne.io.read_raw_eeglab(
        str(filepath),
        preload=True,
        verbose="ERROR",
    )

    raw.pick("eeg")

    data = raw.get_data()
    sfreq = raw.info["sfreq"]

    if data.shape[1] < int(sfreq * 4):
        raise ValueError("EEG-запись короче 4 секунд")

    freqs, psd = welch(
        data,
        fs=sfreq,
        nperseg=int(sfreq * 4),
        axis=1,
```

```
    )

    features = {}

    for channel_idx, channel_name in enumerate(raw.ch_names):
        for band_name, (low, high) in BANDS.items():
            mask = (freqs >= low) & (freqs < high)

            if not np.any(mask):
                raise ValueError(
                    f"Нет частот для диапазона {band_name}"
                )

            power = np.mean(psd[channel_idx, mask])

            features[f"{channel_name}_{band_name}"] = np.log10(
                max(power, 1e-30)
            )

    return features, raw

@app.get("/health")
def health():
    return {
        "status": "ok",
        "model": "RandomForest",
        "feature_count": len(feature_columns),
        "samples_available": len(features_df),
    }


@app.post("/predict")
def predict(request: PredictionRequest):
    if request.sample_index < 0 or request.sample_index >= len(features_df):
        raise HTTPException(
            status_code=400,
            detail="sample_index вне диапазона",
        )

    row = features_df.iloc[[request.sample_index]]

    try:
        # Важно: сохраняем порядок признаков обучения
        X = row[feature_columns]

        prediction = int(model.predict(X)[0])
        probabilities = model.predict_proba(X)[0]

        return {
            "sample_index": request.sample_index,
            "prediction": prediction,
            "class_name": classes[prediction],
            "probabilities": {
                str(classes[int(label)]): float(probability)
                for label, probability in zip(
                    model.classes_,
                    probabilities,
                )
            },
            "feature_count": len(feature_columns),
            "source": "precomputed_eeg_features",
        }

    except Exception as exc:
```

```
        raise HTTPException(status_code=500, detail=str(exc))

@app.get("/predict_file")
def predict_file(
    participant_id: str = Query(
        ...,
        pattern=r"^sub-[A-Za-z0-9]+$",
    )
):
    # Ищем запись участника в derivatives
    matches = list(
        DERIVATIVES_DIR.rglob(
            f"{participant_id}_task-eyesclosed_eeg.set"
        )
    )

    if not matches:
        raise HTTPException(
            status_code=404,
            detail=f"EEG-файл не найден для {participant_id}",
        )

    try:
        features, raw = extract_features(matches[0])

        # Берём первый EEG-канал
        signal_data = raw.get_data(picks=[0])[0]

        # Оставляем 600 точек, чтобы график быстро отрисовывался в Unity
        point_count = min(600, len(signal_data))
        indices = np.linspace(
            0,
            len(signal_data) - 1,
            point_count,
            dtype=int
        )
        signal_data = signal_data[indices]

        # Нормализуем сигнал только для отображения на графике
        max_value = np.max(np.abs(signal_data))
        if max_value > 0:
            signal_data = signal_data / max_value

        channel_name = raw.ch_names[0]

        # Восстанавливаем признаки в том же порядке,
        # который использовался при обучении модели.
        X = pd.DataFrame([features]).reindex(
            columns=feature_columns
        )

        if X.isna().any().any():
            missing = X.columns[X.isna().any()].tolist()

            raise ValueError(
                f"Не удалось вычислить признаки: {missing}"
            )

        prediction = int(model.predict(X)[0])
        probabilities = model.predict_proba(X)[0]

        return {
            "participant_id": participant_id,
            "prediction": prediction,
```

```
            "class_name": classes[prediction],
            "probabilities": {
                str(classes[int(label)]): float(probability)
                for label, probability in zip(
                    model.classes_,
                    probabilities,
                )
            },
            "feature_count": len(feature_columns),
            "channel_count": len(raw.ch_names),
            "duration_seconds": float(raw.times[-1]),
            "source": "openneuro_set_file",
            "channel_name": channel_name,
            "signal": signal_data.tolist()
        }

    except HTTPException:
        raise
    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=str(exc),
        )
```

model_train.py:

```
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import mne

from scipy.signal import welch
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    cross_val_score,
)
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
)
from joblib import dump

warnings.filterwarnings("ignore", category=RuntimeWarning)
mne.set_log_level("ERROR")

# УКАЖИ ПУТЬ К КОРНЕВОЙ ПАПКЕ ДАТАСЕТА
DATASET_DIR = Path(r"C:\Users\faral\Downloads\v1.0.0")

PARTICIPANTS_FILE = DATASET_DIR / "participants.tsv"
DERIVATIVES_DIR = DATASET_DIR / "derivatives"

# Частотные диапазоны ЭЭГ, Гц
BANDS = {
    "delta": (1, 4),
    "theta": (4, 8),
    "alpha": (8, 13),
    "beta": (13, 30),
    "gamma": (30, 45),
}


def extract_features(filepath):
    """Извлекает спектральные признаки из одной EEG-записи."""

    raw = mne.io.read_raw_eeglab(
        str(filepath),
        preload=True,
        verbose="ERROR",
    )

    # Берём только EEG-каналы
    raw.pick("eeg")

    data = raw.get_data()
    sfreq = raw.info["sfreq"]

    if data.shape[1] < int(sfreq * 4):
        raise ValueError("Запись слишком короткая")

    # Спектральная плотность мощности
    freqs, psd = welch(
        data,
```

```
        fs=sfreq,
        nperseg=int(sfreq * 4),
        axis=1,
    )

    features = {}

    for channel_idx, channel_name in enumerate(raw.ch_names):
        for band_name, (low, high) in BANDS.items():
            mask = (freqs >= low) & (freqs < high)

            if not np.any(mask):
                raise ValueError(
                    f"Нет частот для диапазона {band_name}"
                )

            # Логарифм мощности стабилизирует масштаб признаков
            power = np.mean(psd[channel_idx, mask])
            features[f"{channel_name}_{band_name}"] = np.log10(
                max(power, 1e-30)
            )

    return features


def main():
    participants = pd.read_csv(
        PARTICIPANTS_FILE,
        sep="\t",
    )

    print("Распределение участников:")
    print(participants["Group"].value_counts())

    # Оставляем только пациентов с AD и здоровую контрольную группу
    participants = participants[
        participants["Group"].isin(["A", "C"])
    ].copy()

    X_rows = []
    y_rows = []
    participant_ids = []

    for _, participant in participants.iterrows():
        pid = participant["participant_id"]

        # Ищем предварительно обработанную запись
        matches = list(
            DERIVATIVES_DIR.rglob(
                f"{pid}_task-eyesclosed_eeg.set"
            )
        )

        if not matches:
            print(f"[ПРОПУСК] Не найден файл derivatives: {pid}")
            continue

        try:
            features = extract_features(matches[0])

            X_rows.append(features)
            y_rows.append(
                1 if participant["Group"] == "A" else 0
            )
```

```
            participant_ids.append(pid)

            print(f"[OK] {pid}: группа {participant['Group']}")

        except Exception as exc:
            print(f"[ОШИБКА] {pid}: {exc}")

if not X_rows:
    raise RuntimeError(
        "Не удалось обработать ни одной записи. "
        "Проверь путь и структуру derivatives."
    )

X = pd.DataFrame(X_rows).replace(
    [np.inf, -np.inf], np.nan
)

# Удаляем признаки, которые не удалось вычислить
X = X.dropna(axis=1)

y = np.asarray(y_rows)

print("\nРазмер матрицы признаков:", X.shape)
print("Участников в выборке:", len(y))
print("AD:", int(np.sum(y == 1)))
print("Контрольная группа:", int(np.sum(y == 0)))

if len(np.unique(y)) != 2 or np.min(np.bincount(y)) < 2:
    raise RuntimeError(
        "Недостаточно участников одной из групп для обучения."
    )

# Каждый участник представлен одной строкой:
# окна одного человека не попадают одновременно в train и test.
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

model = RandomForestClassifier(
    n_estimators=300,
    class_weight="balanced",
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1,
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("\n=== Результаты на отложенной выборке ===")
print("Accuracy:", round(accuracy_score(y_test, predictions), 3))
print(
    "Balanced accuracy:",
    round(balanced_accuracy_score(y_test, predictions), 3),
)

print("\nОтчёт классификации:")
print(
    classification_report(
```

```
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["CN (контроль)", "AD (Альцгеймер)"],
        zero_division=0,
    )
)

print("Матрица ошибок:")
print(confusion_matrix(y_test, predictions, labels=[0, 1]))

# Дополнительная проверка с разделением по участникам.
# Это оценка на небольшой выборке, а не клиническая валидация.
min_class_count = np.bincount(y).min()
n_splits = min(5, int(min_class_count))

if n_splits >= 2:
    cv = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=42,
    )

    scores = cross_val_score(
        model,
        X,
        y,
        cv=cv,
        scoring="balanced_accuracy",
        n_jobs=1,
    )

    print("\n=== Кросс-валидация ===")
    print("Balanced accuracy по фолдам:", np.round(scores, 3))
    print("Среднее:", round(scores.mean(), 3))
    print("Стандартное отклонение:", round(scores.std(), 3))

# Обучаем итоговую модель на всех доступных участниках
model.fit(X, y)

output_dir = DATASET_DIR / "model_output"
output_dir.mkdir(exist_ok=True)

dump(
    {
        "model": model,
        "feature_columns": X.columns.tolist(),
        "bands": BANDS,
        "classes": {0: "CN", 1: "AD"},
    },
    output_dir / "eeg_model.joblib",
)

X.assign(
    participant_id=participant_ids,
    label=y,
).to_csv(
    output_dir / "eeg_features.csv",
    index=False,
)

print("\nМодель сохранена:")
print(output_dir / "eeg_model.joblib")
print("Признаки сохранены:")
```

```
    print(output_dir / "eeg_features.csv")


if __name__ == "__main__":
    main()
```